

Course Outline: Most Representative Cases in Telemetry

Title: Most Representative Cases in Telemetry **Skill:** 42 — Identificar oportunidades de recolección de datos en escenarios reales que permitan optimizar compañías **Learnpack:** + Casos más representativos en telemetría **File:** s42-w14d42-casos-mas-representativos-en-telemetria **Prerequisite:** Telemetry Architecture Fundamentals (Skill 42 — batch/stream, throttling, serialization) **Target Audience:** Beginner developers in a full-stack AI bootcamp who understand basic data concepts **Total Lessons:** 13 **Format:** Conceptual (100% conceptual — 11 📖 + 1 🗣️ + 1 outro)

Course Description

This course takes students on a tour of how the world's most successful companies use telemetry to make better decisions. Students analyze five domains — product/UX, infrastructure and security, physical world, and business — through real case studies from Spotify, Netflix, Google, Cloudflare, Tesla, Waymo, Amazon, Stripe, and Airbnb. By the end, students can identify the Measure-Analyze-Optimize loop that underlies every example and recognize telemetry opportunities in their own systems.

Course Philosophy

This course emphasizes:

- Pattern recognition over memorization: every example connects back to the same underlying loop
 - Real scale over toy examples: the data volumes and decision speeds discussed are real
 - Breadth as a foundation: understanding the landscape prepares students to implement their own telemetry in Skill 43
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Product Telemetry	2
02 - Systems and Security Telemetry	3
03 - Physical World Telemetry	2
04 - Business Intelligence Telemetry	2
05 - Assessment	2
06 - Outro	1

Section	Lessons
Total	13

00.0 Welcome to Telemetry in the Real World

Learning Objectives:

- Understand what this course covers and why it matters before implementing telemetry in Skill 43
- Identify the five domains where telemetry drives significant business decisions
- Recognize that telemetry is not an advanced topic — it is already shaping most products students use daily

Content Outline:

- The core problem: companies that measure well outcompete those that don't
- What "representative cases" means: real signals, real scale, real decisions
- Preview of the five domains: product, infrastructure/security, physical world, business
- How this course connects to the previous lesson on telemetry architecture and the next on frontend implementation

Transitions: This course starts where the telemetry architecture lesson ended. You now know the mechanics (batch vs stream, serialization, client vs server processing). This course answers the question: what are companies actually doing with it, and why?

Key Principles:

- Telemetry is decision infrastructure — its value comes from the decisions it enables, not the data itself
- Every company in this course collects data to close a feedback loop: signal → analysis → decision → new signal
- The same pattern appears across wildly different domains

Examples:

- A Spotify skip is just a 0 or 1 in a database, but millions of them became one of the world's best music recommendation systems
 - A single Tesla temperature reading is noise, but 500 million readings per day became the ability to predict battery failure before it happens
-

01.0 Product Telemetry

Learning Objectives:

- Understand what product telemetry is and how it differs from backend observability
- Identify the types of behavioral signals that streaming platforms collect
- Recognize why event-level granularity matters for product decisions

Content Outline:

- What is product telemetry: signals generated by user interactions with a product
- The core question product telemetry answers: "What are users actually doing, and what does that tell us about the product?"
- Event-level vs aggregate telemetry: why individual events matter, not just totals
- Preview of the case studies: Spotify (engagement signals) and Netflix (viewing behavior)

Transitions: Product telemetry is the most visible form of telemetry for frontend developers. The next lesson zooms into two world-class implementations from Spotify and Netflix.

Key Principles:

- Product telemetry captures intent, not just actions — a skip means something different at second 5 vs second 55
- The signal must be granular enough to answer specific product questions
- Raw events are stored; analysis happens later — collection and analysis are separate concerns

Examples:

- A "play" event with a timestamp and track ID is the raw signal; the derived insight ("users skip within 10 seconds if they don't recognize the song") requires analysis
- Netflix's per-episode abandonment rate is an aggregate derived from millions of individual pause/stop events

01.1 Engagement Signals in Streaming

Learning Objectives:

- Explain how Spotify uses event-level engagement data to improve its recommendation algorithm
- Explain how Netflix uses viewing behavior telemetry to make content investment decisions
- Distinguish between real-time signal use and batch analysis of historical patterns

Content Outline:

- Spotify's signals: skip (at what second?), replay, failed search, playlist add, share
- What Spotify's algorithm learns: implicit feedback without asking users to rate anything
- The scale: hundreds of millions of daily active users generating billions of micro-events per day
- Netflix's signals: time-to-first-play, per-episode completion rate, pause patterns, rewatch rate
- What Netflix decides from this: which content to renew, cancel, or greenlight next
- The insight: a show with high completion but low rewatch value gets treated differently than one with moderate completion but obsessive rewatching

Transitions: Product telemetry answers questions about user behavior. The next section shifts to a different problem: what happens when your systems, not your users, start generating signals that need monitoring.

Key Principles:

- Implicit feedback (behavior signals) is more honest than explicit feedback (ratings) — users vote with their actions

- The same event (a skip) carries different meaning depending on its context (when, what came before, what the user did next)
- Telemetry replaces opinion with evidence in product decisions: "users prefer shorter episodes" becomes a data claim, not a guess

Examples:

- Spotify found that users who discover music through search are more likely to churn than users who discover through playlists — a finding only possible with event-level telemetry
- Netflix's decision to release all episodes of House of Cards season 1 at once was informed by binge-viewing data from previous licensed content

Anti-pattern — Vanity Metrics: Measuring play count instead of completion rate sounds useful but hides whether users actually engaged. A video with 1 million plays and 10% completion is performing worse than one with 100,000 plays and 80% completion. Telemetry that measures starts instead of meaningful engagement produces decisions that optimize for the wrong thing.

02.0 Systems and Security Telemetry

Learning Objectives:

- Distinguish product telemetry (user behavior) from systems telemetry (infrastructure and security)
- Understand why infrastructure and security monitoring require different approaches to telemetry
- Preview the specific tools and concepts covered in this section (SRE, error budgets, OpenTelemetry, SIEM, EDR)

Content Outline:

- The shift from user-facing to system-facing signals: what changes and what stays the same
- Infrastructure telemetry: is my system healthy? Is it meeting its promises?
- Security telemetry: is my system under attack? What happened after the fact?
- The scale difference: system telemetry is often 10-100x higher volume than product telemetry
- Preview: Google's SRE model, OpenTelemetry as the emerging standard, Cloudflare's DDoS defense

Transitions: Where product telemetry answers "what are users doing?", systems telemetry answers "is my system working, and is it safe?" The next two lessons explore the state of the art in each area.

Key Principles:

- Systems telemetry is often generated automatically — no user action required
- The same infrastructure can generate product telemetry (user requests) and systems telemetry (latency, error rates) simultaneously
- Real-time analysis is often required: a DDoS attack or a cascading failure cannot wait for a batch job

Examples:

- Every HTTP request to Google Search generates both product telemetry (what did the user search?) and systems telemetry (how long did the response take? which server responded? was there an error?)
 - Cloudflare processes 50 million HTTP requests per second — that number itself is the product of systems telemetry infrastructure
-

02.1 SRE, Error Budgets, and OpenTelemetry

Learning Objectives:

- Explain what a Service Level Agreement (SLA) and error budget are and how they interact
- Calculate a basic error budget from an SLA
- Understand why OpenTelemetry emerged and what problem it solves

Content Outline:

- Google's Site Reliability Engineering (SRE): reliability is a feature, not a side effect
- Service Level Agreements (SLAs): a public promise about uptime (e.g. 99.9% = 8.76 hours downtime/year)
- Service Level Objectives (SLOs): internal reliability targets that protect the SLA
- Error budgets: the quantification of acceptable unreliability
 - 99.9% SLA = 0.1% allowed downtime = 8.76 hours per year = 43.8 minutes per month
 - Error budgets turn "is this risky?" into "how much budget does this deployment consume?"
- How error budgets drive decisions: if budget is depleted, stop shipping features and focus on stability
- The observability problem that preceded OpenTelemetry: every vendor had incompatible instrumentation
- OpenTelemetry: a CNCF standard that unifies traces, metrics, and logs under one API
 - Traces: a request's full journey through distributed services
 - Metrics: numerical measurements over time (latency, error rate, request count)
 - Logs: timestamped text records of events

Transitions: Error budgets quantify acceptable downtime for infrastructure. The next lesson looks at a different systems challenge: what happens when the threat is external and deliberate, not accidental.

Key Principles:

- An SLA without telemetry to measure it is a guess, not a commitment
- Error budgets make reliability a shared product decision, not just an ops problem
- Standardization (OpenTelemetry) matters at scale: you cannot afford to migrate instrumentation every time you change vendors

Examples:

- Google's SRE teams have the authority to stop shipping features when the error budget is consumed — telemetry creates the accountability that makes this decision data-driven
- A 99.99% SLA leaves only 52 minutes of downtime per year — less than a single scheduled maintenance window

02.2 Real-Time Security Telemetry 📖

Learning Objectives:

- Explain how Cloudflare uses network telemetry to mitigate DDoS attacks in real time
- Define SIEM and EDR and the type of telemetry each relies on
- Understand why security telemetry requires different latency requirements than product telemetry

Content Outline:

- Cloudflare's scale: 50 million HTTP requests per second → every request is a potential signal
- How DDoS detection works: baseline normal traffic patterns → detect anomalies → block at the edge
- The telemetry that makes this possible: source IP, request rate, headers, payload size, response codes — all in real time
- SIEM (Security Information and Event Management): aggregates logs from firewalls, servers, endpoints, network devices
 - Purpose: correlate events across systems to identify attack patterns humans would miss
 - Example: a failed login on server A + a port scan from the same IP on server B + unusual outbound traffic = probable breach
- EDR (Endpoint Detection and Response): continuous telemetry from individual devices
 - Captures process execution, file writes, network connections, memory access
 - Enables threat hunting: proactively searching for indicators of compromise in historical data

Transitions: Security telemetry operates on digital systems. The next section shifts to the physical world, where telemetry has to deal with the constraints of sensors, bandwidth, and hardware.

Key Principles:

- Security telemetry must be analyzed faster than an attacker can act — latency requirements are measured in milliseconds
- Correlation across multiple telemetry streams is what makes SIEM powerful — no single stream tells the full story
- The volume of security telemetry is too large for humans to read — pattern recognition at machine speed is not optional

Examples:

- Cloudflare's anycast network means that an attacker trying to overwhelm a single data center is automatically distributed across hundreds — telemetry is what coordinates the global response
- A ransomware attack that deploys at 2 AM looks identical to a legitimate IT admin task — EDR telemetry distinguishes them by baseline behavioral patterns, not just signatures

03.0 Physical World Telemetry 📖

Learning Objectives:

- Understand what distinguishes physical-world telemetry from software telemetry
- Identify the types of sensors used in connected vehicles and logistics systems
- Recognize the unique constraints of physical telemetry: data volume, transmission cost, irreversibility

Content Outline:

- What "physical world telemetry" means: sensors on physical objects (vehicles, machines, packages) generating continuous data streams
- Key sensor types: GPS, accelerometers, temperature sensors, LiDAR, cameras, radar, RFID
- The three unique constraints:
 1. Volume: a modern vehicle generates 25GB of data per hour
 2. Transmission cost: not all of it can be sent to the cloud — edge processing decisions matter
 3. Irreversibility: you cannot replay a physical event (a crash, a component failure) to collect more data
- Preview: connected vehicles (Tesla, Waymo, Uber) and logistics (DHL)

Transitions: Software telemetry captures clicks and errors. Physical telemetry captures what's happening in the real world. The decisions it enables are different — and so are the stakes.

Key Principles:

- Edge processing (decide on the device before sending to the cloud) is often necessary when data volumes exceed transmission capacity
- Predictive telemetry (detecting a failure before it happens) is more valuable than reactive telemetry (recording what failed)
- Physical telemetry creates safety obligations: Tesla, Waymo, and Uber all use this data to make decisions that affect human safety

Examples:

- Tesla doesn't send all sensor data to the cloud — it processes most of it on the vehicle and sends compressed summaries plus flagged anomalies
- A DHL package sensor that reports "exceeds 30°C" triggers a human intervention that couldn't happen without that telemetry

03.1 Connected Vehicles and Logistics

Learning Objectives:

- Explain how Tesla uses fleet telemetry for predictive maintenance and OTA updates
- Explain how Waymo, Uber, and DHL use physical telemetry for operational decisions
- Understand the concept of predictive maintenance and why it is more valuable than scheduled maintenance

Content Outline:

- Tesla's fleet telemetry pipeline:
 - Data collected: battery voltage, cell temperature, braking force, motor current, range estimation errors
 - Decision: if 0.1% of fleet shows an unusual battery degradation pattern → push OTA update before other vehicles fail
 - The scale: 5 million vehicles, each reporting continuously, enables statistical detection of failure modes that would never appear in a small test fleet
- Predictive maintenance: detect anomalies in telemetry before they become failures
 - Contrast with scheduled maintenance (every 10,000 km): scheduled maintenance is based on averages, not your specific vehicle's condition
- Waymo's autonomous driving telemetry:
 - LiDAR, cameras, radar, and GPS generate terabytes per hour of driving
 - Every edge case (unusual pedestrian behavior, novel road markings) is flagged and fed back into model training
 - The car improves not just from its own experience but from every other Waymo vehicle's experience
- Uber's real-time route optimization:
 - GPS + live traffic data + driver behavior patterns → ETA that updates every 30 seconds
 - Driver behavior telemetry: hard braking, acceleration patterns → safety scoring
- DHL's fleet and package telemetry:
 - Fleet: location, fuel consumption, speed, idle time → route optimization and emissions reporting
 - Package: temperature, humidity, shock sensors → "last mile" exception handling

Transitions: Physical-world telemetry drives operational and safety decisions. The final content section looks at how telemetry drives commercial decisions — pricing, fraud detection, and conversion optimization.

Key Principles:

- Fleet data (data from thousands of identical devices) enables detection of failure modes no individual device could reveal
- OTA updates powered by telemetry turn product improvement into a continuous process, not a release cycle
- Physical telemetry creates legal and safety accountability — Waymo and Tesla's telemetry is also evidence in liability cases

Examples:

- Tesla's 2019 battery management OTA update (reducing maximum charge on certain Model S vehicles) was triggered by telemetry showing unusual thermal behavior in a subset of battery packs — not by a field failure
- Waymo has driven over 20 million miles on public roads, generating a training dataset no traditional automaker can replicate in a lab

Learning Objectives:

- Understand how behavioral and transactional telemetry drives commercial decisions
- Distinguish business intelligence telemetry from product/UX telemetry
- Identify the types of decisions that funnel analysis, fraud detection, and dynamic pricing enable

Content Outline:

- What "business intelligence telemetry" means: data collected primarily to optimize commercial outcomes (conversion, revenue, fraud prevention, pricing)
- The distinction from product telemetry: product telemetry asks "is the product good?", BI telemetry asks "is the business working?"
- Three use cases this section covers:
 1. Funnel optimization: tracking the path from visit to purchase (Amazon)
 2. Fraud detection: identifying fraudulent transactions in real time (Stripe)
 3. Dynamic pricing: adjusting prices based on observed demand patterns (Airbnb)
- Preview: Amazon (funnel), Stripe (fraud), Airbnb (pricing)

Transitions: Business intelligence telemetry is where data collection creates direct, measurable commercial value. The next lesson shows what that looks like in three world-class implementations.

Key Principles:

- BI telemetry closes the loop between user behavior and business outcome — it makes the cost of a bad UX visible in revenue terms
- Real-time BI telemetry (Stripe's fraud detection) requires the same low-latency infrastructure as security telemetry
- Dynamic pricing is only fair if the pricing model is based on actual demand data, not guesses

Examples:

- Amazon estimates that a 100ms improvement in page load time is worth \$1.6 billion per year in revenue — this number comes from telemetry connecting latency to conversion
- Stripe processes trillions of dollars in transactions per year; without real-time fraud telemetry, the fraud rate would be economically catastrophic

04.1 Funnels, Fraud, and Dynamic Pricing

Learning Objectives:

- Explain how Amazon uses funnel telemetry to optimize conversion
- Explain how Stripe uses transaction pattern telemetry to detect fraud in milliseconds
- Explain how Airbnb uses observed demand signals for dynamic pricing

Content Outline:

- Amazon's funnel telemetry:
 - Tracked at every step: search query, product views, add-to-cart, checkout start, purchase

- Drop-off analysis: where do users abandon the funnel? What changed when drop-off rates shifted?
- Personalization signals: scroll depth on a product page, image vs description engagement, review reading patterns
- A/B testing at scale: Amazon runs thousands of experiments simultaneously, each measured by conversion rate
- Stripe's fraud detection:
 - Signals: transaction amount, merchant category, geographic location, device fingerprint, transaction velocity, time of day
 - Baseline: a cardholder in Madrid who makes 3-5 transactions per day suddenly making 30 transactions in 4 different countries in 1 hour = anomaly
 - Stripe's ML models score every transaction in under 100ms — the card is either approved or declined before the user sees a response
 - The feedback loop: flagged transactions that turn out to be fraud train the model; false positives (legitimate transactions declined) are also logged and used to reduce false positive rate
- Airbnb's dynamic pricing:
 - Signals: listing views, booking rate, search volume for the area, historical seasonal patterns, local events, competitor pricing
 - Airbnb's "Smart Pricing" product suggests price adjustments in real time based on aggregated demand signals
 - The incentive alignment problem: hosts want maximum revenue, guests want fair prices — Airbnb's pricing telemetry has to serve both

Transitions: You've now seen 9 companies across 4 domains. The next lesson extracts the pattern they all share.

Key Principles:

- Funnel telemetry makes invisible friction visible — you cannot fix what you cannot measure
- Fraud detection at scale requires machine learning precisely because the volume of signals exceeds human review capacity
- Dynamic pricing is only as fair as the data it's based on — telemetry quality directly determines pricing quality

Examples:

- Amazon's "1-Click Buy" feature was invented after funnel telemetry showed that checkout friction was the single biggest driver of cart abandonment
- Stripe's Radar fraud detection system is sold as an add-on product — the telemetry Amazon and Netflix generate in-house, Stripe monetizes as a service

05.0 Cross-Domain Patterns

Learning Objectives:

- Extract the Measure-Analyze-Optimize loop from all five domains studied in this course

- Identify where each company's telemetry system fits in the loop
- Apply the pattern to a new domain to evaluate whether a telemetry opportunity exists

Content Outline:

- The loop that appears in every case study:
 1. Signal: something observable happens (a skip, a packet, a temperature reading, a cart abandonment)
 2. Collection: the signal is captured and transmitted (event stream, sensor feed, log aggregation)
 3. Analysis: patterns are extracted from many signals (real-time or batch)
 4. Decision: the analysis enables a better decision than was possible without the data
 5. Feedback: the decision creates new conditions that generate new signals
- Mapping each case study to the loop:
 - Spotify: implicit feedback loop — skip → algorithm update → different songs → more data on new choices
 - Google SRE: reliability loop — error → budget consumed → shipping freeze → stability improves
 - Cloudflare: threat loop — anomaly detected → traffic blocked → attacker adapts → new anomaly detected
 - Tesla: maintenance loop — thermal anomaly → OTA update → new baseline → next anomaly detected against new baseline
 - Amazon: conversion loop — cart abandonment → friction identified → UX change → new conversion rate measured
- What breaks the loop: missing signals, stale analysis, decisions not fed back into the system

Transitions: This pattern is what you'll implement in the next learnpack. Before that, test your understanding of each domain.

Key Principles:

- The value of telemetry is not in the data itself but in the loop it closes
- Faster loops (real-time) are not always better — the right latency depends on how fast the decision needs to be made
- A broken feedback loop (signals collected but not analyzed, or analysis done but decisions not made) is a common and expensive failure mode

Examples:

- A company that collects telemetry but never builds dashboards or feeds it to ML models is paying the cost of telemetry without any of the benefits
- Google's error budget only works because the signal (downtime measured against SLA) feeds directly into a deployment decision — remove the decision and the loop breaks

05.1 Telemetry Case Studies 🧠

Learning Objectives:

- Apply the Measure-Analyze-Optimize loop to new scenarios
- Identify the right telemetry signals for specific business questions
- Distinguish between telemetry that enables real-time decisions and telemetry that enables strategic decisions

Question Format: Scenario-based

Questions:

1. A video streaming service wants to understand why subscribers cancel within the first 30 days. They have basic login and play-count data. What additional telemetry signals should they collect, and what specific questions would each signal help answer?
2. An e-commerce company sets a 99.95% SLA for its checkout service. How many minutes of downtime does their monthly error budget allow? If the service has been down for 18 minutes this month, how much budget remains, and what should the team do about non-critical deployments for the rest of the month?
3. A financial services company is seeing unusual transaction patterns on weekend nights. Their current fraud detection system flags transactions manually after customer reports. Based on Stripe's approach, what telemetry signals would you recommend collecting, and at what latency should analysis run?
4. A logistics company wants to reduce fuel costs and late deliveries across its fleet of 2,000 vehicles. Inspired by DHL's approach, what physical telemetry signals would you instrument, and what decisions would each signal enable?
5. An online marketplace is considering dynamic pricing for its listings. What demand signals would you collect (inspired by Airbnb's approach), and what anti-patterns should you avoid to ensure the pricing remains fair?
6. A developer says: "We added telemetry to our app six months ago. We collect 50 events per user session. But we haven't changed anything in the product since then." Using the Measure-Analyze-Optimize loop, identify where the feedback loop is broken and what the team should do to fix it.

Evaluation Criteria:

- Q1-5: Does the student identify specific, actionable signals (not generic "collect more data")? Does each signal connect clearly to a decision?
- Q6: Does the student correctly identify that collection without analysis or decisions is a broken loop, not a telemetry success?

Score Interpretation:

- 5-6 correct: Strong understanding of telemetry as decision infrastructure across domains
 - 3-4 correct: Good foundation; review the specific domains where signals were unclear
 - 0-2 correct: Return to the case studies and focus on the signal → decision connection in each one
-

06.0 Outro

Content Outline:

- Course recap: what students accomplished
 - Analyzed 9 companies across 4 domains (product, infrastructure/security, physical world, business)
 - Extracted the Measure-Analyze-Optimize loop underlying every case
 - Learned to identify telemetry opportunities and the signals that enable specific decisions
- Connection to bigger picture
 - This course was reconnaissance — understanding what telemetry looks like in production prepares you to build it yourself
 - The next learnpack (Skill 43) begins the implementation journey: what to collect from your own frontend, why each signal matters, and how to model the data
- Next steps and continued learning
 - OpenTelemetry documentation: the emerging standard for instrumentation
 - Google's SRE Book (free online): the definitive resource on reliability engineering
 - Stripe's engineering blog: real-world case studies on fraud detection and ML at scale
- Encouragement
 - The gap between "we collect some logs" and "we have a telemetry-driven product" is exactly what you're learning to close

Note: This is conclusion only — no theory, exercises, or assessment.